

ASP.NET Core Email Confirmation

Example Confirmation Link

`https://localhost:44304/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-4400ba886550&token=CfDJ8Hpirs`

action (userId;token)
query string parameters

Generate Email Confirmation Token

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Generate Email Confirmation Link

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",  
    new { userId = user.Id, token = token }, Request.Scheme);
```

action controller
http, https

Use `UserManager` service `GenerateEmailConfirmationTokenAsync()` method. This method takes one parameter. The `user` for whom we want to generate the email confirmation token.

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Build the email confirmation link

Once we have the `token` generated, build the `email confirmation link`. The user simply clicks this link to confirm his email. This link executes, `ConfirmEmail` action in `Account` controller. The `user ID` and the `email confirmation token` are passed in the query string. Model binding in ASP.NET core maps the values from the query string parameters to the respective parameters on the `ConfirmEmail` action.

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",  
    new { userId = user.Id, token = token }, Request.Scheme);
```

The generated `confirmation link` would look like the following

```
https://localhost:44304/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-4400ba886550&token=CfDJ8Hpirs
```

The last parameter `Request.Scheme` returns the request protocol such as `Http` or `Https`. This parameter is required to generate the full absolute URL. If this parameter is not specified, a relative URL like the following will be generated.

```
/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-4400ba886550&token=CfDJ8Hpirs
```

```

[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid) after registration of a new user, confirm email before login
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City
        };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded) generate token
        {
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                { userId = user.Id, token = token }, Request.Scheme);

            if (signInManager.IsSignedIn(user) && user.IsInRole("Admin"))

```

Once we get this confirmation link, we want to email it to the user for confirmation, so he can login to his email service provider and click on it to confirm his email. We will discuss sending emails in our upcoming videos. What we will do instead in this video is log this confirmation link to a file. For that we need to inject BuildInLogger service.

```

ment
EmployeeManagement.Controllers.AccountController
AccountController(UserManager<Ap
references
public class AccountController : Controller

private readonly UserManager<ApplicationUser> userManager;
private readonly SignInManager<ApplicationUser> signInManager;
private readonly ILogger<AccountController> logger;

0 references
public AccountController(UserManager<ApplicationUser> userManager,
    SignInManager<ApplicationUser> signInManager,
    ILogger<AccountController> logger)
{
    this.userManager = userManager;
    this.signInManager = signInManager;
    this.logger = logger;
}

```

log the confirmation link:

```
logger.Log(LogLevel.Warning, confirmationLink);
```

```
EmployeeManagement.Controllers.AccountController Register(RegisterViewModel model)

if (result.Succeeded)
{
    var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

    var confirmationLink = Url.Action("ConfirmEmail", "Account",
        { userId = user.Id, token = token }, Request.Scheme);

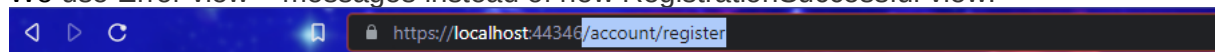
    logger.Log(LogLevel.Warning, confirmationLink);

    if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
    {
        return RedirectToAction("ListUsers", "Administration");
    }
    //confirm_before_login
    //await signInManager.SignInAsync(user, isPersistent: false);
    return RedirectToAction("index", "home");

    ViewBag.ErrorTitle = "Registration successful";
    ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
        "email, by clicking on the confirmation link we have emailed you";
    return View("Error");
}
```

```
ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");
```

We use Error view + messages instead of new RegistrationSuccessful view.



An unhandled exception occurred while processing the request.

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered.

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)

Stack Query Cookies Headers Routing

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered.

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateEmailConfirmationTokenAsync(TUser user)

EmployeeManagement.Controllers.AccountController.Register(RegisterViewModel model) in AccountController.cs

73. var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

It can't find any token provider, that can generate token. To fix this, we need to provide a token provider - default from Asp.net core or our own custom.

Add Default Token Providers

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddIdentity<ApplicationUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>()
        .AddDefaultTokenProviders();
}
```

.AddDefaultTokenProviders();

```
EmployeeManagement.Startup
services.AddIdentity<ApplicationUser, IdentityRole>()
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;

    options.SignIn.RequireConfirmedEmail = true;
}.AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders();
```

```
var user = new ApplicationUser
{
    UserName = model.Email,
    Email = model.Email,
    City = model.City,
};

var result = await userManager.CreateAsync(user, model.Password);

if (result.Succeeded)
{
    var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

    var confirmationLink = Url.Action("ConfirmEmail", "Account",
        new { userId = user.Id, token = token }, Request.Scheme);

    logger.Log(LogLevel.Warning, confirmationLink);

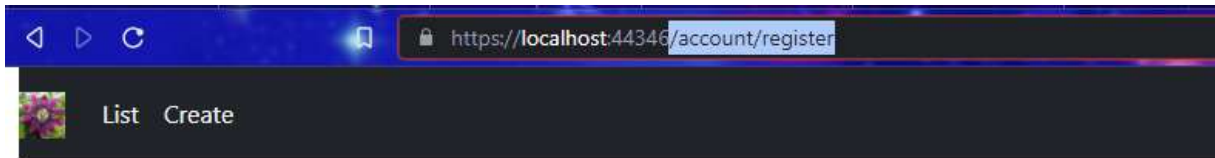
    if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
    {
        return RedirectToAction("ListUsers", "Administration");
    }

    ViewBag.ErrorTitle = "Registration successful";
    ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
```

```

    "email, by clicking on the confirmation link we have emailed you";
    return View("Error");
}

```



Registration successful

Before you can Login, please confirm your email, by clicking on the confirmation link we have emailed you

dbo.AspNetUsers [Data] AccountController.cs Startup.cs							
Max Rows: 1000							
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	Password
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAAE

We can find the confirmation link from nlog.config file -> *.log

```

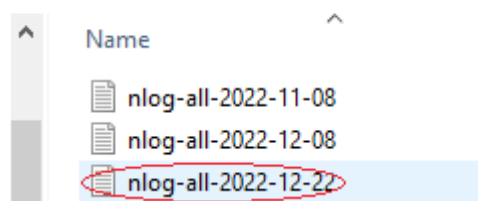
nlog.config | AccountController.cs | Startup.cs
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <!-- the targets to write to -->
  <targets>
    <!-- write logs to file -->
    <target name="allfile" xsi:type="File"
            fileName="c:\DemoLogs\nlog-all-${shortdate}.log"/>
  </targets>

  <!-- rules to map from logger name to target -->
  <rules>
    <!--All logs, including from Microsoft-->
    <logger name="*" minlevel="Trace" writeTo="allfile" />
  </rules>
</nlog>

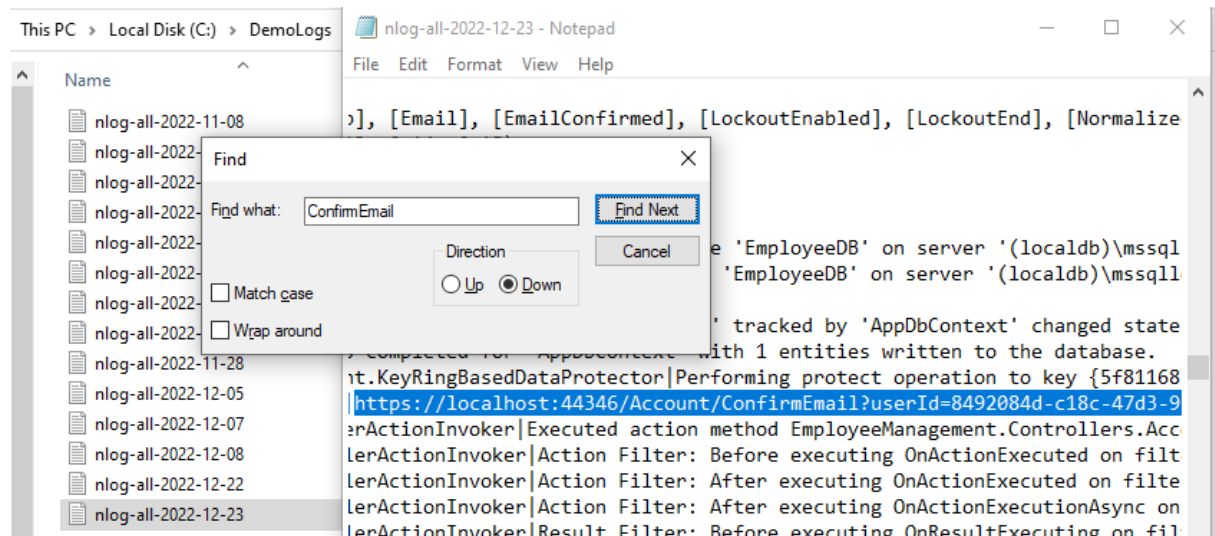
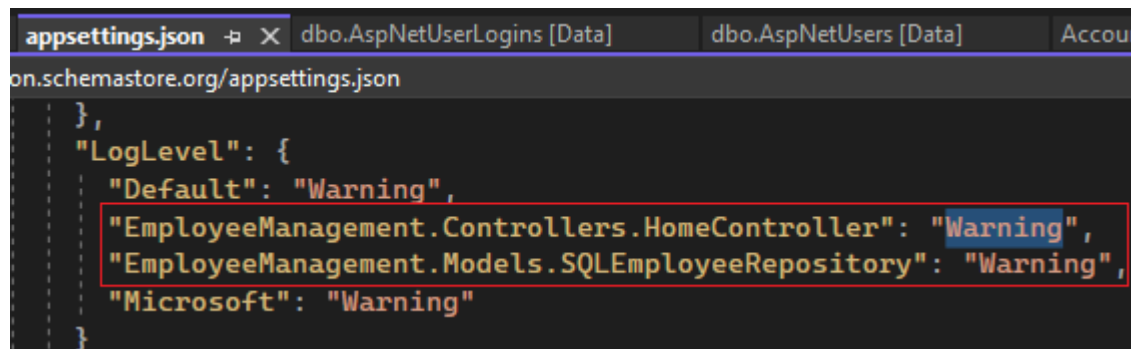
```

This PC > Local Disk (C:) > DemoLogs



I can't find the confirmation link in my version, even searching for "ConfirmEmail" action.

```
POST http://localhost:44346/account/register application/x-www-form-urlencoded 279 - 200 - text/
GET http://localhost:44346/_framework/aspnetcore-browser-refresh.js - -
M45D3V32MG" completed keep alive response.
GET http://localhost:44346/_framework/aspnetcore-browser-refresh.js - - - 200 12006 application/
GET http://localhost:44346/_vs/browserLink - -
M45D3V32MH" completed keep alive response.
GET http://localhost:44346/_vs/browserLink - - - 200 - text/javascript;+charset=UTF-8 17.4961ms
M45D3V32MG" disconnecting.
M45D3V32MH" disconnecting.
M45D3V32MG" stopped.
M45D3V32MH" stopped.
id "0HMN45D3V32MH" sending FIN because: "The Socket transport's send loop completed gracefully."
id "0HMN45D3V32MG" sending FIN because: "The Socket transport's send loop completed gracefully."
```



<https://localhost:44346/Account/ConfirmEmail?userId=8492084d-c18c-47d3-9bcf-cdbe57a6e1f2&token=CfDJ8IcWgV9y4PJks6bemgk2FP4WyyeS2qx2651rbQBgdvqgs4Hk1VJ8tWW%2BqVxlvNMSpsJPqVX5I3qgqQOqnS8CbJCqGHRr3I%2Bt8tqiwfUy%2BxwD6p9I9OSzFwwGfyTXbCrZr2qyJwlbRzhmXm0khLtwmrqu2Q2oV2%2FyfVtaccXZAQMvYzh%2BXTpG4gBFeJzOyYYNk3qnczcwJGXgQoLYmUIH1McszdbGBaZXw4TqkKvrDUgsTTi0SBtjq1E3plhvnkQA%3D%3D>

-> 404 because we don't have ConfirmEmail action, so we implement it.

```

EmployeeManagement.Controllers.AccountController ConfirmEmail(string userI
[AllowAnonymous]
0 references
public async Task<IActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    store
    var user = await userManager.FindByIdAsync(userId);
    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);
    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}

```

View/Account -> new razor view ConfirmEmail.cshtml:

<h3>Thank you for confirming your email**</h3>**

When we reload the ConfirmLink,
we are logged in and the value in Users DB table is set to true.

Without this link, the other users are not confirmed yet.

Register Action in Account Controller

```
[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> Register(RegisterViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City,
        };

        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                new { userId = user.Id, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, confirmationLink);

            // Rest of the code
        }
    }
}
```

ConfirmEmail Action - Account Controller

```
[AllowAnonymous]
public async Task<IActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    var user = await userManager.FindByIdAsync(userId);
    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);
    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}
```